Normalizers::Body. But a class usually gets *instance* methods by including a module—not *class* methods. How can a mixin like Normalizers::Body bend the rules and define class methods on its includer?

Look for the answer in the definition of the Body module itself:

gems/vcr-2.5.0/lib/vcr/structs.rb

```
module VCR
  module Normalizers
    module Body
      def self.included(klass)
        klass.extend ClassMethods
      end

      module ClassMethods
        def body_from(hash_or_string)
          # ...
```

The code above pulls off a convoluted trick. Body has an inner module named ClassMethods that defines body_from and other methods as regular instance methods. Body also has an included *Hook Method (157)*. When Request includes Body, it triggers a chain of events:

- Ruby calls the included hook on Body.
- The hook turns back to Request and extends it with the ClassMethods module.
- The extend method includes the methods from ClassMethods in the Request's singleton class. (You might remember this last part of the trick from *Quiz: Module Trouble, on page 129*.)

As a result, body_from and other instance methods get mixed into the singleton class of Request, effectively becoming class methods of Request. How's that for a complicated code concoction?

This ClassMethods-plus-hook idiom used to be quite common, and it was used extensively by the Rails source code. As you'll see in Chapter 10, *Active Support's Concern Module, on page 179*, Rails has since moved to an alternate mechanism—but you can still find examples of the idiom in VCR and other gems.

Quiz: Checked Attributes (Step 5)

Where you finally earn Bill's respect and the title of Master of Metaprogramming.

The following is the code that we wrote in the previous development step: